

Software Engineering Group
Heidelberg University

Prof. Dr. Sebastian Baltes
Tobias Reichel

Advanced Software Engineering (IASE)
Assignment 6

Event Sourcing

Submitted by:

Rohit Sonejee Matrikelnummer: 4756184

Thanh Tran Matrikelnummer: 4209012

Group 18

Summer Semester 2026

July 13, 2026

Declaration of GenAI Use

We, **Rohit Sonejee** (ID 4756184) and **Thanh Tran** (ID 4209012), declare that we used generative AI tools in the development of this assignment. Specifically, we used **Gemini** and **Claude**. Below is a transparent breakdown of what was done independently versus with AI assistance.

Parts Completed Independently

- Designing the revert functionality for the event-sourced persistence layer, including the compensation logic for reverting `INSERT`, `UPDATE`, and `DELETE` events while preserving the append-only event log.
- Integrating the revert feature into the existing hexagonal architecture by extending the domain ports, services, and API endpoints while maintaining the separation between the `api`, `domain`, and `data` modules.
- Implementing and validating the optimistic locking workflow using the exposed `version` field and ensuring the required HTTP responses (200, 204, 404, and 409) matched the assignment specification.
- Testing the implementation using the provided Gradle test suite and verifying that the project continued to satisfy formatting, static analysis, architecture, and coverage requirements.
- Researching and understanding event sourcing and the CRUD approach as well as writing the reflection report (`REPORT.md`), including the discussion of event sourcing tradeoffs, use cases, and snapshots.

AI-Assisted Parts

- Gemini and Claude were used as pair-programming assistants to review Kotlin code, identify potential implementation issues, and suggest refactorings during the development of the revert functionality.
- Used AI to help debug compilation errors, review implementation correctness.
- Used AI for grammar and spelling corrections.
- Assisted with the general drafting and formatting of this LaTeX report.

Exercise 6.1: Event Sourcing - Revert Implementation

In this exercise, the `revert` logic was implemented using event sourcing. The changes were validated against the project's strict compilation, style, and test coverage requirements.

Build Output Validation

Figure 1 demonstrates that the project compiles successfully, passing all 155 tests and satisfying all Detekt, Ktlint, and JaCoCo coverage gates.

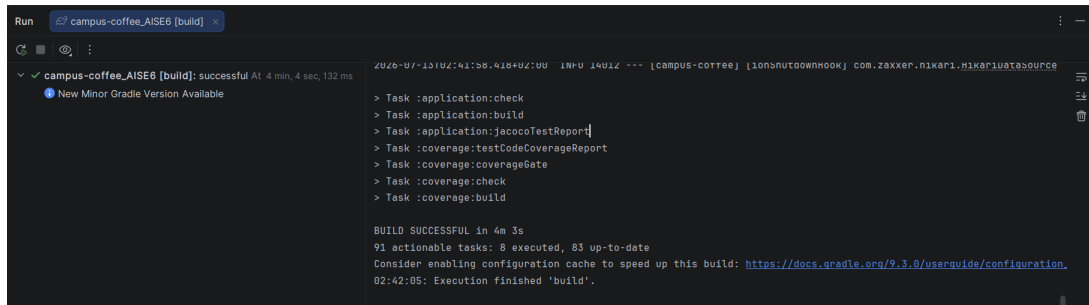
A screenshot of an IDE's Run window showing the output of a Gradle build for 'campus-coffee_AISE6'. The build is successful, taking 4 minutes and 4 seconds. The output lists tasks: application:check, application:build, application:jacocoTestReport, coverage:testCodeCoverageReport, coverage:coverageGate, coverage:check, and coverage:build. It also shows 'BUILD SUCCESSFUL in 4m 3s' and '91 actionable tasks: 8 executed, 83 up-to-date'. A link to the Gradle documentation is provided at the bottom.

Figure 1: Successful Gradle build output showing 100% test completion and coverage.

API Functionality

Figure 2 shows the Swagger UI documentation confirming that the `POST /api/pos/{id}/revert` endpoint has been successfully exposed and integrated into the system.



Figure 2: Swagger UI showing the newly implemented Revert endpoint for Points of Sale.

Exercise 6.2: Reflection

(a) Tradeoffs

When comparing event sourcing with the plain relational adapter in CampusCoffee, the conventional CRUD approach relies on overwriting the current state directly in the database. To handle concurrent modifications safely, a relational adapter often must utilize a **pessimistic locking mechanism** (or optimistic locking) to prevent conflicting updates. Event sourcing sidesteps in-place updates entirely by appending immutable events to a log, guaranteeing that the true history is preserved perfectly without losing previous states.

For what kinds of projects or requirements does event sourcing justify its added complexity?

Event sourcing justifies its added complexity in projects where we may need to secure us legally (by keeping track of changes done) or if we want to be able to reconstruct past states.

When does a conventional approach serve better?

It's better when we build a very simple project (where the history plays no role) or for example if we only have limited time to develop a project.

(b) Use cases

1. We can see who approved a review by inspecting the events table which stores `ReviewApproval` events. `EventRepository` and `EventEntity` return approver ID and timestamp for entries written by `EventSourcedReviewApprovalDataService`.
2. `EventStore` records create, update and delete events for reviews. `EventSourcedReviewDataService` and `ReadModelProjector` let us reconstruct who created, edited or deleted a review and when.

(c) Snapshots

As proposed in this exercise, we could periodically make snapshots. Then we can rebuild from the newest snapshot and only have to replay the events that come after the snapshot.